

Relatório do Trabalho Prático

Implementação de uma aplicação gráfica 3D inspirada na temática
'Retro Gaming' recorrendo à biblioteca Three.js

Elementos do Grupo:

Filipe Silva al82239

Liane Duarte al79012

Pedro Braz al81311

Docentes:

Miguel Melo | Bruno Peixoto | Guilherme Gonçalves | Luís Barbosa

Índice

1. Introdução	3
2. Pesquisa de Aplicações Semelhantes	4
3. Arquitetura e Organização do Projeto	5
4. Implementação dos Requisitos	7
4.1. Construção de Objetos 3D Complexos	7
4.2. Configuração de Camaras.....	10
4.3. Configuração de Luzes.....	12
4.4. Interação com a Cena	14
4.5. Animação	16
5. Funcionalidades Adicionais	18
5.1. Sistema de Menus e Garagem	18
5.2. Áudio com Web Audio API	19
5.3. Painel de Controlo lil-gui	20
6. Conclusão	21

1. Introdução

A temática imposta pelo protocolo do Trabalho Prático de Computação Gráfica para o ano letivo 2025/2026, *Retro Gaming*, oferece um ponto de partida rico para explorar os conceitos abordados ao longo do semestre. Os jogos das primeiras gerações *arcade*, com a sua estética abstrata e dependência de geometrias simples, encontram em ferramentas modernas como a biblioteca *Three.js* um meio expressivo natural, podendo ser reinterpretados em três dimensões sem perder a identidade visual original.

O jogo escolhido foi *Tron Light Cycles* (Bally Midway, 1982), um *arcade* inspirado no filme homónimo da Disney lançado no mesmo ano. A escolha justifica-se pela presença sólida do jogo no cânone cultural do *retro gaming*, pela forte adequação da sua estética *neon* ao paradigma do *WebGL*, e pelo facto de a sua mecânica permitir demonstrar quase todos os conceitos exigidos pelo protocolo de forma natural, desde a construção de objetos 3D complexos até à animação contínua e interação em tempo real.

A aplicação desenvolvida, denominada **NEON DRIVE**, reinterpreta em 3D a mecânica de *Tron Light Cycles* em quatro arenas temáticas distintas (*SPACE*, *DESERT*, *JUNGLE* e **ICE/LAVA**), cada uma com decoração, iluminação e atmosfera próprias. Após seleccionar a arena e o veículo numa garagem dedicada, o utilizador controla a sua viatura através do teclado, deixando atrás de si um rasto de luz que constitui o núcleo da mecânica competitiva. O sistema suporta múltiplos modos de câmara, iluminação configurável por *toggle* individual, áudio gerado pela *Web Audio API* e um painel de controlo *lil-gui*.

O presente relatório organiza-se da seguinte forma: o Capítulo 2 apresenta a pesquisa de aplicações semelhantes; o Capítulo 3 documenta a arquitetura do projeto; o Capítulo 4 detalha a implementação de cada um dos cinco requisitos do protocolo, com referência aos conceitos teóricos das aulas; o Capítulo 5 cobre funcionalidades adicionais; o Capítulo 6 sintetiza conclusões e aponta melhorias futuras.

2. Pesquisa de Aplicações Semelhantes

Antes de iniciar o desenvolvimento, foram pesquisadas aplicações que partilham conceitos ou mecânicas com o trabalho proposto. Esta análise permitiu identificar referências visuais, escolhas técnicas comuns e oportunidades de diferenciação. As quatro aplicações selecionadas cobrem desde o jogo *arcade* original até implementações modernas em 3D e variantes que partilham apenas o conceito central de movimento contínuo com rasto.

Tron (Bally Midway, 1982) é o jogo *arcade* original que deu origem a toda a linhagem. Construído sobre uma estética de gráficos vetoriais a duas dimensões, apresenta uma vista de topo (*top-down*) e divide-se em quatro minijogos, sendo o *Light Cycles* aquele que serviu de base direta ao **NEON DRIVE**. A sua mecânica simples (mover, virar em ângulos retos e evitar paredes de luz) e a paleta de cores vivas sobre fundo preto definiram um padrão estético que persiste até hoje. Do jogo original foi mantido o conceito-base de paredes de luz crescentes e a paleta cromática de alto contraste, mas a vista *top-down* foi substituída por uma abordagem totalmente tridimensional com múltiplos modos de câmara.

GLtron é uma reinterpretação 3D *open-source* do *Light Cycles*, considerada uma das referências mais diretas para implementações modernas do conceito. Mantém uma arena plana com grelha visível, motas estilizadas e adversários controlados por inteligência artificial. Constituiu a inspiração mais próxima na fase inicial do **NEON DRIVE**, particularmente no que diz respeito ao posicionamento da câmara e ao desenho da arena. A diferenciação assenta na introdução de quatro biomas temáticos (*SPACE*, *DESERT*, *JUNGLE* e *ICE/LAVA*) com decoração e iluminação próprias, na alternância entre veículos e numa garagem com personalização de cores.

Armagetron Advanced é também um projeto *open-source*, com forte componente multijogador *online* e uma comunidade competitiva ativa. Apresenta arenas em 3D, suporte para mapas personalizados e diversos modos de jogo. Embora a vertente *online* não tenha sido objetivo do **NEON DRIVE**, o seu desenho de arena e a forma como organiza a interface visual foram úteis para validar decisões sobre o nivelamento de iluminação e a leitura clara das paredes de luz em movimento. O **NEON DRIVE** distingue-se por privilegiar uma experiência local, com maior ênfase na variedade visual entre arenas e na qualidade estética de cada cenário.

Slither.io é um jogo *online* gratuito com uma mecânica conceptualmente próxima, embora em duas dimensões e com estética muito mais simples: o jogador controla uma serpente que cresce continuamente e colide consigo própria ou com adversários. A relevância para este trabalho não reside em aspetos técnicos ou

visuais, mas no estudo da própria mecânica de rasto crescente enquanto elemento central de jogabilidade, particularmente no equilíbrio entre liberdade de movimento e penalização por colisão. Esta análise reforçou a decisão de manter o controlo do veículo simples e direto no **NEON DRIVE**, deixando que a complexidade emerja do ambiente tridimensional e não dos comandos.

3. Arquitetura e Organização do Projeto

A complexidade da aplicação, com quatro arenas distintas, viaturas seleccionáveis, sistema de menus, áudio e múltiplos modos de câmara, exigiu desde cedo uma organização modular que evitasse a concentração de lógica num único ficheiro. O projeto foi estruturado de forma que cada componente tenha uma responsabilidade bem delimitada, facilitando o desenvolvimento em paralelo entre os elementos do grupo e simplificando a deteção de problemas.

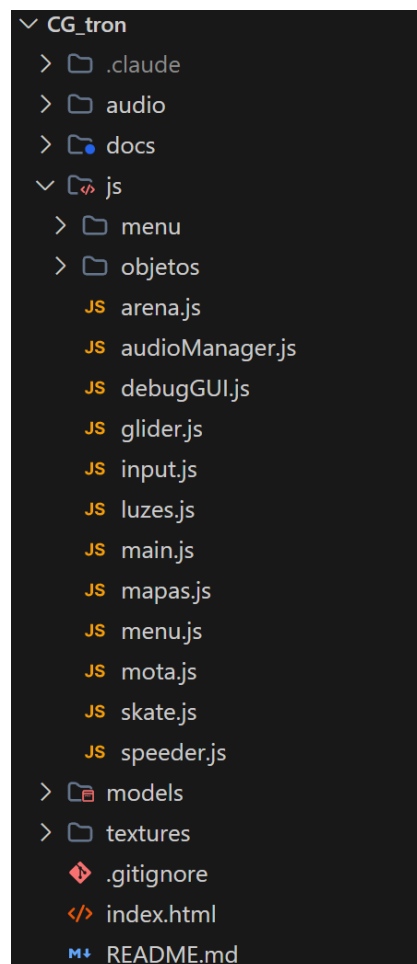


Figura 1 - Arquitetura do trabalho

Como ilustra a Figura 1, na raiz do projeto encontram-se o ficheiro `index.html`, porta de entrada da aplicação responsável por carregar a biblioteca *Three.js* via *importmap* e o *script* principal, e três pastas de recursos: `audio/`, com os ficheiros MP3 das bandas sonoras de menu e arena; `models/`, com os modelos externos importados nos formatos GLTF, OBJ e MTL; e `textures/`, organizada por bioma e por viatura, contendo as imagens de difusão, *normal map* e *roughness*. Existe ainda uma pasta `docs/` com a documentação interna do projeto e um ficheiro `README.md` na raiz com as instruções de execução e visão geral.

A pasta `js/` agrupa toda a lógica do jogo, com os ficheiros principais distribuídos por responsabilidade. O `main.js` constitui o motor: cria a cena, as câmaras e o *renderer*, e gere o *loop* de animação, decidindo qual o mapa a carregar com base na seleção feita no menu. O `arena.js` é responsável pelas fundações comuns a todas as arenas, nomeadamente o plano do chão, a grelha *neon* (GridHelper) e a abóbada de estrelas. O `mapas.js` funciona como dicionário de configurações, definindo para cada bioma a sua paleta cromática, intensidade de nevoeiro (*fog*) e conjunto de texturas.

A iluminação global da cena foi extraída para um módulo dedicado, `luzes.js`, que centraliza a criação das fontes de luz partilhadas por todos os mapas (luz ambiente, luz direcional, luz pontual da arena e duas luzes pontuais associadas às viaturas) e ajusta as suas cores, intensidades e configurações de sombra em função do mapa selecionado. Este ficheiro expõe ainda uma função `toggleLuz()` que permite ativar e desativar cada fonte individualmente, suportando o requisito de iluminação configurável definido no protocolo. Em paralelo, o módulo `input.js` agrupa o tratamento de teclado e rato, mantendo o estado dos controlos e desacoplando-o da lógica de jogo, e o `audioManager.js` gere o *crossfade* entre os temas musicais e os efeitos sonoros gerados por osciladores. As viaturas têm cada uma o seu ficheiro próprio, `mota.js` e `skate.js`, com a respetiva construção geométrica e parametrização visual.

Por fim, o `debugGUI.js` constrói o painel de controlo *lil-gui* visível no canto superior direito durante o jogo, agregando os *toggles* e *sliders* que expõem em tempo real os parâmetros de iluminação, áudio, ambiente e modo de câmara. Esta separação assegura que toda a lógica de interface de depuração fica isolada e pode ser desativada ou removida sem afetar o resto do sistema.

A subpasta `js/objetos/` foi criada para isolar a decoração específica de cada bioma e evitar a poluição do *core*. Contém quatro ficheiros, um por arena (`arenaDeserto.js`, `arenaGelo.js`, `arenaJungle.js` e `arenaSpace.js`), seguindo todos o mesmo contrato: cada módulo exporta uma função `adicionarObjetosX()`, que desenha a decoração e adiciona iluminação local específica do bioma (focos sobre

objetos, luzes embutidas em cristais, partículas luminosas), e uma função `atualizarX(delta)`, chamada a cada *frame* para animar os elementos dinâmicos. Esta uniformidade permite que o `main.js` lide com qualquer arena de forma genérica, bastando consultar o identificador do bioma selecionado para invocar o par de funções correto.

A subpasta `js/menu/` agrupa o sistema de interface do utilizador, organizado em torno de uma máquina de estados (`menuState.js`) que gere as transições entre os ecrãs principal, da garagem (`garage.js`) e de seleção de mapa. O ficheiro `menu.js`, presente na raiz da pasta `js/`, serve de ponto de entrada para este subsistema e é invocado diretamente a partir do `main.js`.

O fluxo da aplicação segue uma sequência linear bem definida: o utilizador inicia no ecrã principal, navega para a garagem para escolher viatura e cor, transita para o ecrã de seleção de mapa, e entra no *loop* de jogo. Em qualquer momento, a tecla **ESC** permite voltar ao menu, devolvendo o controlo à máquina de estados.

4. Implementação dos Requisitos

O presente capítulo descreve a forma como cada um dos cinco requisitos definidos no protocolo do trabalho prático foi implementado. Cada sub-secção inicia com um enquadramento teórico que retoma os conceitos abordados nas aulas e relevantes para o requisito em questão, prossegue com a descrição da implementação concreta e termina com a apresentação dos resultados visuais obtidos.

4.1. Construção de Objetos 3D Complexos

Enquadramento teórico

Em computação gráfica, um objeto 3D é tipicamente representado por uma malha poligonal (*mesh*), constituída por uma coleção de vértices, arestas e faces interligados. A malha poligonal é uma representação de fronteira que descreve a superfície de um objeto sólido através de polígonos, sendo os triângulos a primitiva mais comum por garantirem coplanaridade e simplificarem a rasterização. Objetos 3D complexos podem ser construídos por instanciação e composição hierárquica de primitivas básicas, sendo este precisamente o paradigma adotado pela *Three.js*, que disponibiliza primitivas geométricas (`BoxGeometry`, `CylinderGeometry`, `SphereGeometry`, `OctahedronGeometry`, entre outras) que podem ser combinadas, transformadas e agrupadas para formar modelos arbitrariamente elaborados.

A aparência das superfícies depende dos materiais e texturas que lhes são aplicados. Uma textura é uma imagem 2D mapeada sobre a malha através de coordenadas de textura (s, t) definidas nos vértices e interpoladas pelo rasterizador. Para além da textura difusa, que define a cor base, são frequentemente utilizadas texturas auxiliares como o *normal map* (que altera a normal aparente da superfície para simular relevo sem aumentar a contagem de polígonos), o *roughness map* (que controla quão difusa é a reflexão), o *metalness map* (que indica que regiões se comportam como metal) e o *ambient occlusion map* (que escurece zonas onde a luz indireta tem dificuldade em chegar). A combinação destas texturas, no contexto de um modelo de iluminação fisicamente baseado (PBR), permite obter superfícies com aspeto realista mesmo sobre geometrias relativamente simples.

Implementação

A cena combina três classes distintas de objetos 3D complexos: as viaturas selecionáveis pelo jogador, a decoração específica de cada bioma e modelos externos importados em casos pontuais. As viaturas, em particular, foram construídas integralmente a partir das primitivas básicas exigidas pelo protocolo, cada uma desenvolvida do zero por um elemento do grupo, e constituem o núcleo desta sub-secção.

Mota (Pedro): A *light cycle*, inspirada na estética de *Tron Legacy*, é montada por composição hierárquica sob um THREE.Group e divide-se em nove subsistemas: rodas, suspensão, bloco do motor, carenagem, secção dianteira, traseira e exaustores, guarda-lamas, *cockpit* e iluminação. Cada roda combina um pneu maciço (CylinderGeometry com 48 segmentos, rodada 90° no eixo Z) com um aro *neon* embutido, jante, disco de travão e cubo central, complementados por cinco raios distribuídos angularmente e posicionados através de funções cos/sin. O bloco do motor empilha dez barbatanas finas de arrefecimento ao longo do comprimento e aloja seis células de energia emissivas em cilindros blindados. A carenagem é constituída por placas laterais inclinadas para o interior com frisos *neon* angulares e entradas de ar agressivas, encimadas por um tanque com uma "espinha dorsal" iluminada. A frente apresenta um bico em cunha com faróis *neon* duplos e a traseira concentra um *spoiler* com friso luminoso e duas turbinas cilíndricas que escondem núcleos *neon* a simular a chama da propulsão. O guarda-lamas, que tipicamente requereria uma TorusGeometry, foi reconstruído por sete segmentos de BoxGeometry distribuídos num arco de matemática trigonométrica, criando um efeito facetado deliberadamente cibernético. O *cockpit*, sem piloto orgânico, sugere condução autónoma com vidro fumado, painel HUD em material

emissivo e *clip-ons* com punhos iluminados. Foram aplicados materiais PBR em todas as superfícies, recorrendo a um conjunto completo de cinco mapas para o metal (cor base, *normal*, *roughness*, *metallic* e *ambient occlusion*) e a uma textura de concreto repetida três vezes ao longo dos pneus para sugerir o aspeto rugoso da borracha dura.

hover-skate (Liane): A segunda viatura propõe uma reinterpretação tecnológica do conceito de prancha de *skate*, próxima do *hoverboard* de *Back to the Future Part II* mas com a paleta cromática do NEON DRIVE. A ausência deliberada de rodas é o elemento distintivo: a prancha flutua acima do chão, suportada visualmente por quatro propulsores cilíndricos colocados nos cantos. Cada propulsor é uma micro-construção em si, com carcaça exterior dupla, dois anéis *neon* concêntricos, câmara interna, núcleo emissivo de energia, campo translúcido inferior e quatro braços de fixação radiais distribuídos por trigonometria. A prancha combina um corpo central em BoxGeometry, tampas que simulam o arredondamento das pontas, uma nervura longitudinal de reforço, placas laterais de blindagem e uma malha de frisos *neon* nas arestas, no topo e na zona inferior, esta última a constituir o *underglow* da viatura. Uma rede de condutos de energia (cilindros longitudinais e transversais) atravessa toda a prancha e termina em módulos de processamento posicionados nos cruzamentos. A traseira aloja um motor multi-estágio com bocal central, dois exaustores secundários laterais, núcleos emissivos e aletas em V reforçadas com nervuras e duplo friso *neon*. Sobre o topo, painéis antiderrapantes simulam *grip* e duas zonas marcam a posição dos pés. A iluminação é particularmente rica: para além da luz central de *underglow*, cada um dos quatro propulsores carrega o seu próprio PointLight, complementados por um farol frontal e uma luz traseira de exaustão. O hover-skate inclui ainda uma componente de animação intrínseca, com a prancha a oscilar verticalmente por uma função sinusoidal, os núcleos dos propulsores a rodar continuamente sobre o eixo Y, e a intensidade emissiva dos núcleos e condutos a pulsar com diferentes frequências, conferindo uma sensação de aparelho energizado e vivo.

Veículo (Filipe):

Decoração dos biomas: Para além das viaturas, cada uma das quatro arenas inclui um conjunto extenso de objetos 3D complexos gerados por composição programática a partir de primitivas. Exemplos incluem as falésias estratificadas e a Grande Pirâmide *voxel* da arena DESERT, os glaciares hexagonais e os cristais facetados da arena GELO, as torres ADN com dupla-hélice rotativa e os monólitos da arena SPACE, e as árvores e formações rochosas da arena JUNGLE. Em casos onde o número de instâncias justifica, foi utilizada InstancedMesh para reduzir o número de chamadas à GPU a uma única chamada, mesmo desenhando milhares de cubos.

Modelos importados: Em situações pontuais em que o reconhecimento visual justificava o recurso a modelos externos, foi utilizado o GLTFLoader para carregar a árvore *quiver tree* na arena DESERT, e o par OBJLoader/MTLLoader para o modelo *Oddish* gigante colocado fora dos limites da arena JUNGLE. Em ambos os casos, os modelos foram instanciados, escalados e iluminados especificamente para se integrarem na temática.

Texturização: A maior parte das texturas difusas, *normal* e *roughness* utilizadas nos cenários e nas viaturas foi obtida do repositório aberto *Poly Haven*. As paredes e elementos *neon* recorrem a materiais emissivos (MeshStandardMaterial com a propriedade *emissive* ativa e *toneMapped*: false) que brilham por si só, independentemente da iluminação da cena, sustentando assim a estética visual característica do jogo.

Colocar imagens dos objetos feitos por cada um

4.2. Configuração de Camaras

Enquadramento teórico

Numa aplicação gráfica 3D, a câmara virtual é o elemento que determina o que é projetado no ecrã, por analogia direta com o funcionamento de uma câmara fotográfica convencional. Conforme abordado nas aulas, o sistema de visualização opera sobre três sistemas de coordenadas distintos: o sistema do mundo (*World Coordinates System*, WCS), no qual os objetos da cena são posicionados; o sistema da câmara (*View Reference Coordinates*, VRC), centrado na câmara e orientado segundo os seus vetores de referência (*View Reference Point*, *View Plane Normal* e *View Up Vector*); e, finalmente, o sistema de coordenadas do dispositivo (*Device Coordinates*, DC), que corresponde aos *pixels* efetivamente desenhados no ecrã. A *pipeline* de visualização 3D encadeia as transformações que convertem os vértices entre estes sistemas, recorrendo a quatro matrizes principais: a matriz de visualização, a matriz de projeção, a divisão perspectiva e a matriz de *viewport*.

A escolha do tipo de projeção define a aparência fundamental da imagem. A **projeção em perspectiva** simula o comportamento da visão humana e da maior parte dos dispositivos óticos: o volume de visualização tem a forma de um tronco de pirâmide invertida (*frustum*) cujo vértice coincide com a posição da câmara, e os raios de projeção convergem nesse ponto, fazendo com que objetos mais distantes apareçam menores. É parametrizada pelo campo de visão (*field-of-view*, FOV), pelo *aspect ratio*, e pelos planos de recorte anterior e posterior, que delimitam o que é efetivamente desenhado. A **projeção ortográfica**, em contraste,

utiliza raios de projeção paralelos, definindo o volume de visualização por um paralelepípedo. Como a profundidade não afeta o tamanho aparente dos objetos, esta projeção é particularmente útil para vistas técnicas, mapas e perspectivas de topo, onde a leitura espacial precisa de ser independente da distância.

Implementação

O NEON DRIVE suporta três modos distintos de câmara, alternáveis em tempo real através da tecla **C** ou pelo painel *lil-gui*, todos eles construídos sobre a *PerspectiveCamera* e a *OrthographicCamera* da *Three.js*, classes que abstraem a construção das matrizes descritas acima a partir dos parâmetros do utilizador. A escolha de qual câmara está ativa é gerida no *main.js* e é diretamente alimentada ao *renderer.render(cena, câmara)* em cada *frame*.

O modo **terceira pessoa** é o predefinido durante o jogo e utiliza uma *PerspectiveCamera* com FOV de 60° e *aspect ratio* dinâmico ajustado à largura da janela. A câmara é posicionada atrás e ligeiramente acima da viatura ativa, sendo a sua posição atualizada a cada *frame* em função da posição e orientação do veículo, com uma componente de interpolação suave que evita movimentos bruscos quando o jogador vira. Este modo é o que melhor preserva a sensação imersiva do jogo, mantendo o jogador sempre orientado em relação ao rasto de luz que vai deixando.

O modo **livre** recorre também à *PerspectiveCamera*, mas associa-lhe um controlador *OrbitControls* que permite ao utilizador orbitar à volta da arena, aproximar ou afastar e inclinar o ângulo de visão recorrendo ao rato. É o modo mais útil para inspecionar visualmente a cena, demonstrar a riqueza de detalhe dos cenários, e validar a iluminação durante a defesa do trabalho.

O modo **topo** utiliza uma *OrthographicCamera* posicionada diretamente sobre o centro da arena e orientada para baixo, oferecendo uma vista aérea sem distorção perspetiva. Esta vista é particularmente útil para perceber a topologia da arena na sua totalidade, ler a posição relativa das viaturas e antecipar trajetórias, ecoando o estilo *top-down* do *arcade* original de 1982 e cumprindo simultaneamente o requisito do protocolo de demonstrar a alternância entre projeção perspetiva e ortográfica.

A tecla **C** percorre ciclicamente os três modos, sendo o estado atual também refletido no painel *lil-gui*, onde o utilizador pode selecionar diretamente o modo desejado a partir de uma lista. Em todos os modos, o redimensionamento da janela do *browser* é detetado e desencadeia a atualização do *aspect ratio* da *PerspectiveCamera* e dos limites do volume da *OrthographicCamera*, garantindo que a imagem se mantém correta em qualquer resolução.

4.3. Configuração de Luzes

Enquadramento teórico

A iluminação é o que confere volume, profundidade e atmosfera a uma cena 3D, e o seu cálculo divide-se em duas componentes complementares: o **modelo de iluminação**, que descreve a natureza e distribuição da luz que incide sobre as superfícies, e o **modelo de reflexão**, que descreve como essa luz é devolvida para a câmara em função das propriedades de cada material. O modelo de reflexão de **Phong**, abordado nas aulas, decompõe a cor final num ponto da superfície na soma de três componentes: a componente **ambiente**, que ilumina indiretamente todos os objetos com uma intensidade uniforme e evita que zonas em sombra fiquem completamente pretas; a componente **difusa**, calculada pela lei de Lambert como o produto interno entre a normal à superfície e a direção da fonte de luz, e responsável pela noção de volume nas superfícies foscas; e a componente **especular**, que modela o brilho concentrado em torno da direção de reflexão ideal e cuja extensão depende do grau de polimento do material. A **aproximação de Blinn**, que substitui o cálculo do vetor de reflexão pelo do vetor *halfway*, permite implementar esta última componente de forma computacionalmente mais eficiente, dando origem ao modelo **Blinn-Phong** que constitui a base da iluminação em tempo real na maior parte das aplicações gráficas.

A *Three.js* implementa estes conceitos através de uma família de classes de luz, cada uma simulando um tipo distinto de fonte. A *AmbientLight* corresponde diretamente à componente ambiente, aplicando uma intensidade uniforme e omnidirecional a toda a cena. A *DirectionalLight* simula uma fonte infinitamente distante, com raios paralelos, sendo a analogia mais próxima da luz solar; é também a fonte mais usada para projetar sombras, porque a sua direção constante simplifica o cálculo do *shadow map*. A *PointLight* simula uma fonte pontual que irradia em todas as direções, como uma lâmpada, e está sujeita a atenuação atmosférica em função da distância. A *SpotLight* corresponde a um holofote, restringindo a emissão a um cone de abertura controlada. Por fim, a *HemisphereLight* é uma fonte ambiente composta, que mistura uma cor para o hemisfério superior e outra para o hemisfério inferior, simulando a interação entre a luz do céu e a do solo.

Implementação

A iluminação do NEON DRIVE foi estruturada em duas camadas complementares: uma camada **global**, comum a todas as arenas e centralizada num módulo dedicado, e uma camada **local**, específica de cada bioma e definida nos respetivos ficheiros da pasta `js/objetos/`. Esta separação permite que cada arena tenha uma identidade luminosa própria sem duplicar as fontes essenciais, mantendo simultaneamente os parâmetros globais facilmente ajustáveis num único ponto.

A camada global é construída pela função `criarLuzes(cena, mapa)` no ficheiro `luzes.js`, e instala cinco fontes principais: uma `AmbientLight` que estabelece o nível mínimo de iluminação da cena, uma `DirectionalLight` que serve de fonte principal e projeta as sombras, uma `PointLight` central da arena, e duas `PointLight` adicionais que acompanham as viaturas dos jogadores e contribuem para o efeito de *underglow*. As cores e intensidades destas fontes são depois ajustadas em função do identificador do mapa selecionado: na arena DESERT, a `DirectionalLight` adquire um tom alaranjado intenso (`0xFFB347`) com elevada intensidade, simulando o sol forte de um deserto; na arena JUNGLE, a luz direcional toma um tom amarelo-esverdeado e intensidade reduzida, evocando luz filtrada pela folhagem; na arena GELO, a `AmbientLight` é escurecida e a `DirectionalLight` torna-se uma luz branca extremamente intensa, criando o contraste dramático típico de um ambiente nevado; e na arena SPACE, a iluminação direcional é reduzida ao mínimo, deixando que sejam os objetos emissivos da própria cena (torres ADN, monólitos, drone vigia) a fornecer a maior parte da luz.

As sombras são calculadas pelo algoritmo *Percentage-Closer Filtering* (PCF) na sua variante suavizada (`PCFSoftShadowMap`), com a `DirectionalLight` configurada com `castShadow: true` e parâmetros de *shadow camera* ajustados a cada arena. O tamanho do *shadow map* foi calibrado em função da exigência visual de cada bioma: a arena GELO recebe 2048×2048 para garantir nitidez nas sombras dos cristais e dos glaciares, enquanto as restantes utilizam 1024×1024 por equilíbrio entre qualidade e desempenho. Os limites do volume ortográfico da câmara de sombra são também ajustados ao tamanho de cada arena para evitar tanto artefactos de quantização (causados por um volume demasiado largo) como sombras cortadas (causadas por um volume demasiado estreito).

A camada local de iluminação reside dentro de cada ficheiro `arenaX.js` e adiciona fontes específicas aos objetos do bioma. Na arena DESERT, cada Torre de Glifos possui duas `PointLight` amarelas (uma na base e outra no topo) que iluminam os hieróglifos das paredes e o solo em redor, e cada Torre de Cristal possui uma `PointLight` central de elevada intensidade no cume e luzes menores embutidas em cada andar e em cada cápsula da escadaria em espiral. Na arena GELO, uma

HemisphereLight global simula uma aurora boreal com tons de azul, complementada por uma PointLight carmesim no extremo da arena que rasga o fundo azulado com um brilho dramático, e cerca de 10% dos pilares de gelo recebem aleatoriamente uma PointLight ciano de baixo para cima. Na arena JUNGLE, uma SpotLight colocada a 40 unidades de altura e apontada ao centro da arena simula um feixe de luz a perfurar o dossel da floresta. Na arena SPACE, cada Torre ADN combina uma PointLight ciano na base com uma PointLight magenta no topo, replicando as cores das partículas instanciadas, e cada monólito do fundo emite uma PointLight central de alta intensidade que ilumina os pequenos cubos animados nas suas paredes.

A função toggleLuz(luzes, tipo) exposta pelo luzes.js permite ativar ou desativar individualmente cada uma das cinco fontes globais, atribuindo false à propriedade visible da luz selecionada. Esta função é invocada a partir do painel *lil-gui*, onde *checkboxes* dedicados expõem o estado de cada luz ao utilizador, cumprindo o requisito do protocolo. As fontes locais permanecem ativas para preservar a identidade visual de cada bioma, mas o utilizador pode usar os toggles globais para isolar visualmente o efeito de cada componente sobre a cena.

4.4. Interação com a Cena

Enquadramento teórico

Uma aplicação gráfica interativa distingue-se de uma aplicação não-interativa por dar ao utilizador um papel ativo na geração das imagens, recebendo *input* em tempo real e propagando o seu efeito para a cena antes da próxima ronda do *pipeline* gráfico. O modelo concetual de um sistema gráfico interativo, abordado nas aulas, organiza esta dinâmica em torno de dois fluxos: a **transformação de saída**, que converte a representação interna da cena na imagem que o utilizador vê no ecrã, e a **transformação de entrada**, que recolhe os eventos dos dispositivos de *input* e os converte em alterações ao estado da aplicação. Os dispositivos de *input* podem ser classificados em **físicos** (teclado, rato, *gamepad*, ecrã tátil) e **lógicos** (eventos de seleção, de localização, de valoração, de escolha entre alternativas), sendo papel da aplicação fazer a ponte entre uns e outros através da escuta apropriada dos eventos do navegador.

Implementação

O NEON DRIVE recolhe e processa *input* em três frentes complementares: o **teclado** durante o jogo, o **rato** no modo de câmara livre e nos menus, e os **controlos do painel *lil-gui*** para ajuste de parâmetros em tempo real. Toda a lógica relativa ao teclado e rato durante o jogo está concentrada no módulo input.js, que

mantém um objeto de estado consultado pelo `main.js` em cada *frame* e que isola completamente a deteção do *input* da aplicação dos seus efeitos sobre a cena.

A **viatura do Jogador 1** é controlada pelas teclas **W**, **A**, **S** e **D**, e a **viatura do Jogador 2** pelas **setas direcionais**, suportando assim modo multijogador local com dois conjuntos de controlos independentes. Em ambos os casos, o eixo longitudinal (W/S e seta para cima/baixo) atua sobre a aceleração no eixo Z local da viatura, e o eixo transversal (A/D e setas esquerda/direita) é convertido numa viragem contínua sobre o eixo Y, simulando a condução de uma motorizada. A tecla **Espaço** desencadeia um salto, modelado como uma trajetória parabólica com gravidade simulada que permite à viatura transpor obstáculos baixos e descolar momentaneamente do plano da arena.

As restantes teclas dão acesso a comandos de sistema. A tecla **C** percorre ciclicamente os três modos de câmara descritos na Sub-secção 4.2 (terceira pessoa, livre e topo). A tecla **R** reinicia a partida, repondo as posições e estados das viaturas e limpando os rastros de luz. A tecla **ESC** devolve o controlo ao menu principal, fazendo a transição de estado correspondente na máquina de estados do subsistema de menus.

O **rato** atua em dois contextos distintos. Nos menus (principal, garagem e seleção de mapa), os botões e elementos interativos da interface são clicáveis e apresentam realce visual ao serem sobrevoados, recorrendo aos eventos `mousemove` e `click` do navegador. No modo de câmara livre durante o jogo, o controlador `OrbitControls` interpreta o arrastar do botão esquerdo como rotação orbital à volta do centro da arena, o arrastar do botão direito como deslocação lateral, e a roda do rato como aproximação ou afastamento, oferecendo seis graus de liberdade na inspeção da cena.

Por último, o painel **lil-gui** constitui um terceiro mecanismo de interação, ortogonal aos anteriores, e está disponível durante todo o jogo no canto superior direito do ecrã. Construído pelo módulo `debugGUI.js`, expõe um conjunto de controlos categorizados em quatro secções: **Iluminação** (intensidade da luz ambiente, cor da luz ambiente, intensidade da direcional, ativação de sombras), **Áudio e Som** (volume da música de fundo e dos efeitos sonoros), **Ambiente** (cor de fundo, ativação de *wireframe* nas paredes da arena, ativação de nevoeiro) e **Câmara** (modo ativo). Cada controlo é vinculado diretamente à propriedade correspondente do objeto que afeta, pelo que qualquer alteração se reflete imediatamente na cena, sem necessidade de reiniciar o jogo. Esta abordagem não só cumpre o requisito de interação configurável e de *toggle* individual das luzes definido no protocolo, como ainda constitui uma ferramenta de demonstração

extremamente útil durante a defesa do trabalho, ao permitir mostrar em tempo real o efeito isolado de cada componente sobre a cena renderizada.

4.5. Animação

Enquadramento teórico

Uma animação em computação gráfica é, no seu cerne, uma sequência de imagens estáticas (frames) apresentadas em rápida sucessão, em que cada imagem corresponde a uma versão ligeiramente alterada da anterior. A ilusão de movimento contínuo emerge da capacidade do sistema visual humano para integrar sequências de imagens com cadências superiores a aproximadamente 24 imagens por segundo, sendo o objetivo prático em aplicações interativas atingir cadências de 60 ou mais para garantir suavidade e responsividade. O motor de animação consiste, portanto, num loop que, a cada frame, atualiza as propriedades dos objetos da cena (posições, rotações, escalas, atributos de geometria, parâmetros de material) e invoca o renderer para produzir a nova imagem.

A manipulação dos objetos é feita através das três **transformações geométricas** fundamentais abordadas nas aulas: **translação**, que desloca o objeto somando um vetor à sua posição; **rotação**, que gira o objeto em torno de um eixo por uma matriz de rotação; e **escala**, que multiplica as suas dimensões por um fator. Em coordenadas homogêneas, as três transformações exprimem-se uniformemente como multiplicações de matrizes 4×4 , e podem ser compostas numa única **matriz composta** que representa simultaneamente várias operações. A ordem de aplicação é relevante, uma vez que a multiplicação matricial não é comutativa, e as transformações associam-se da direita para a esquerda pela ordem inversa de aplicação. Em cenas complexas, é prática comum representar os modelos hierarquicamente através de **grafos de cena**, em que cada nó possui a sua transformação local e propaga-a aos descendentes; ao mover o nó-pai, todos os filhos se deslocam coerentemente, simplificando a animação de objetos compostos.

Um aspeto fundamental para a qualidade visual é o uso de um **delta time** consistente. Como os monitores podem operar a frequências de atualização variáveis (60 Hz, 144 Hz, 240 Hz), animar com base no índice do frame faria com que o ritmo das animações dependesse da frequência do ecrã. A solução é multiplicar todas as variações por um valor delta, que mede o tempo decorrido desde o último frame, garantindo que o movimento por unidade de tempo é o mesmo em qualquer dispositivo.

Implementação

No NEON DRIVE, o motor de animação está localizado no main.js e é construído sobre a função `requestAnimationFrame()` do navegador, que sincroniza a invocação do loop com a taxa de atualização do ecrã. A cada frame, é consultado um objeto `THREE.Clock()` para obter o delta em segundos, valor que é depois propagado a todas as funções de atualização específicas de cada subsistema, nomeadamente as funções `atualizarSpace(delta)`, `atualizarDeserto(delta)`, `atualizarGelo(delta)`, `atualizarJungle(delta)` e `atualizarSkate(delta)`. Cada uma destas funções utiliza uma guarda de validação no início (`if (!_estado) return;`) que ignora a execução caso o subsistema não esteja ativo no mapa atual, poupando ciclos de processamento.

A **animação das viaturas** é o caso mais imediato. Como cada viatura está organizada hierarquicamente sob um `THREE.Group` (princípio do grafo de cena), basta aplicar translação e rotação ao grupo raiz para que toda a estrutura, com as suas dezenas de peças, se mova como uma unidade coerente. O movimento é alimentado pelo estado mantido em input.js: a aceleração calculada pela tecla longitudinal é integrada na velocidade, a velocidade é multiplicada pelo delta e somada à posição, e a viragem é aplicada como rotação contínua sobre o eixo Y local. Quando o utilizador pressiona a tecla **Espaço**, é desencadeada uma trajetória de salto modelada por uma equação parabólica com gravidade constante, fazendo com que a viatura suba até a uma altura máxima e regresse ao plano da arena.

O **hover-skate**, em particular, possui uma componente de animação intrínseca que é independente do input do jogador e está implementada na função `atualizarSkate(delta)`. A prancha oscila verticalmente por uma função sinusoidal $\text{Math.sin}(t * 1.8) * 0.15$, simulando a flutuação característica de um hoverboard; os núcleos dos quatro propulsores rodam continuamente sobre o eixo Y a uma velocidade de $3.0 * \text{delta}$; e a intensidade emissiva tanto dos núcleos como dos condutos de energia da prancha pulsa por funções sinusoidais com diferentes frequências (6 Hz para os núcleos, 4 Hz para os condutos), com um deslocamento de fase distinto em cada conduto para evitar pulsação sincronizada. O conjunto destas pequenas animações confere ao veículo a sensação de estar energizado mesmo em repouso.

A **decoreção de cada arena** é animada pelas funções `atualizarX(delta)` correspondentes. Na arena DESERT, milhares de partículas de areia são deslocadas por funções trigonométricas que simulam o vento, com os atributos de posição armazenados num `BufferGeometry` e a propriedade `geometry.attributes.position.needsUpdate = true` a sinalizar à GPU que é

necessário recarregar os dados. Na arena JUNGLE, esporos e folhas caem em espiral por combinações de seno e cosseno aplicadas sobre o tempo. Na arena GELO, milhares de pequenos cubos de "neve digital" são instanciados via InstancedMesh e rodam erraticamente enquanto descem. Na arena SPACE, as torres ADN possuem grupos de animação que rodam continuamente sobre o eixo Y através de incrementos do tipo `grupo.rotation.y += 1.2 * delta`; o drone vigia central oscila em altura e a sua escolta de pequenos octaedros gira em torno dele; e os lasers verticais são reposicionados ciclicamente quando atingem o limite inferior.

Os **rastos de luz**, núcleo da mecânica de jogo, são gerados continuamente atrás de cada viatura e crescem em comprimento ao longo do tempo. Cada rasto é construído por extrusão progressiva de segmentos BoxGeometry posicionados na trajetória do veículo, com material emissivo na cor neon do jogador. A deteção de colisão é feita verificando, a cada frame, se a posição da viatura intersecta algum dos segmentos existentes, recorrendo a bounding boxes axiais alinhadas para minimizar o custo computacional.

A possibilidade de **rever ou reiniciar** as animações é assegurada pela tecla **R**, que dispara uma função de reset que repõe as posições e velocidades iniciais das viaturas, limpa os rastos de luz acumulados e devolve o tempo de simulação a zero. Esta funcionalidade satisfaz o requisito do protocolo de "repetir/rever a animação" e é simultaneamente útil durante a defesa do trabalho, ao permitir reproduzir a mesma sequência de movimento múltiplas vezes para inspeção.

5. Funcionalidades Adicionais

Para além dos cinco requisitos definidos no protocolo, foram desenvolvidas três componentes que aumentam a complexidade da aplicação, contribuem para a sua originalidade e melhoram a experiência do utilizador. Estas componentes não são exigidas pelo enunciado, mas integram-se naturalmente na arquitetura do projeto e foram tratadas com o mesmo rigor das restantes.

5.1. Sistema de Menus e Garagem

A entrada e saída do *loop* de jogo é mediada por um sistema de menus organizado em torno de uma máquina de estados finitos, implementada no ficheiro

menuState.js. Cada estado corresponde a um ecrã visível pelo utilizador: o **ecrã principal**, com o título *NEON DRIVE* em estética *neon* sobre uma grelha em perspetiva e quatro botões de navegação (jogar, garagem, créditos, sair); a **garagem** (garage.js), onde o utilizador escolhe entre as viaturas disponíveis (mota e *hover-skate*) e personaliza a respetiva cor *neon* a partir de uma paleta pré-definida; e o **ecrã de seleção de mapa**, onde escolhe uma das quatro arenas (SPACE, DESERT, JUNGLE e GELO/LAVA), com cada opção apresentada por uma miniatura representativa e uma cor temática. As transições entre estados são desencadeadas por cliques nos botões correspondentes ou pela tecla **ESC**, e cada uma é acompanhada por uma transição visual suave que melhora a sensação de polimento da aplicação.

A garagem é o ecrã com maior carga de demonstração técnica. A viatura selecionada é renderizada ao centro do ecrã num *canvas* dedicado, com a sua animação intrínseca ativa e iluminação configurada para a destacar, podendo ser inspecionada antes da escolha. A mudança de viatura é instantânea, recriando o objeto com a cor *neon* selecionada e libertando os recursos GPU do anterior através das funções `destruirSkate()` e equivalentes que percorrem o grafo, libertando geometrias, materiais e texturas. Esta gestão cuidada de memória previne fugas de recursos numa aplicação que pode passar por dezenas de transições durante uma sessão.

5.2. Áudio com Web Audio API

A componente sonora da aplicação foi desenvolvida com base na Web Audio API nativa do navegador, escolha que dispensa bibliotecas externas e oferece controlo de baixo nível sobre a geração e mistura de som. A lógica reside no módulo `audioManager.js`, que gere dois tipos distintos de áudio: os **efeitos sonoros (SFX)** das interações com a interface, gerados em tempo real a partir de osciladores matemáticos (`OscillatorNode`) com envelope de amplitude controlado; e a **banda sonora**, carregada de ficheiros MP3 e reproduzida através de nós `AudioBufferSourceNode`. Os SFX gerados por código incluem o som de seleção de botão, o som de confirmação na garagem e o som de início de partida, todos com perfis sonoros distintos obtidos por combinação de frequências, formas de onda (sine, square, triangle) e curvas de decaimento.

A banda sonora suporta **transição suave (crossfade)** entre temas, particularmente útil na transição do menu (com tema synthwave ambiente) para o jogo (com o tema próprio de cada arena). O crossfade é implementado por dois nós `GainNode` em paralelo, cujos parâmetros de ganho são interpolados linearmente

ao longo de aproximadamente um segundo, garantindo que a transição não é cortada nem produz cliques audíveis. O volume tanto da música de fundo como dos efeitos sonoros é exposto no painel lil-gui, permitindo ao utilizador ajustá-los em tempo real ou silenciá-los completamente.

5.3. Painel de Controlo lil-gui

O painel de controlo construído com a biblioteca lil-gui, descrito também na Subsecção 4.4 do ponto de vista da interação, é uma das funcionalidades que mais contribui para o valor demonstrativo do trabalho. Concentrado no canto superior direito do ecrã durante toda a sessão de jogo, agrega num único painel hierárquico os parâmetros de quatro componentes do sistema: iluminação, áudio, ambiente da cena e modo de câmara. Para cada parâmetro, o controlo apropriado é gerado automaticamente pela biblioteca em função do tipo (controlo deslizante para valores numéricos, checkbox para booleanos, seletor de cor para cores, dropdown para enumerações), e a alteração propaga-se imediatamente para a propriedade vinculada do objeto correspondente, sem necessidade de reinicializar o jogo.

Mais importante do que a sua utilidade durante o desenvolvimento, o painel constitui um instrumento de **demonstração ao vivo** durante a defesa do trabalho. Ao permitir desligar individualmente cada fonte de luz, ativar e desativar o nevoeiro, alternar entre câmaras e ajustar volumes em tempo real, dá ao avaliador uma forma direta e tangível de verificar que cada componente do sistema é genuinamente independente e está corretamente implementado, em vez de uma sequência de ecrãs estáticos.

6. Conclusão

O desenvolvimento do **NEON DRIVE** permitiu consolidar na prática os conceitos abordados na unidade curricular e demonstrar o seu funcionamento numa aplicação de complexidade significativa. Os cinco requisitos do protocolo foram cumpridos: foram construídos objetos 3D complexos a partir de primitivas da *Three.js*, com destaque para as duas viaturas desenvolvidas individualmente pelos elementos do grupo; foram implementadas câmaras com projeção perspetiva e ortográfica alternáveis em tempo real; foram instaladas cinco fontes de luz com *toggle* individual; foi desenvolvido um sistema de interação por teclado e rato com suporte multijogador local; e foram desenhadas animações contínuas, periódicas e desencadeadas pelo utilizador, com possibilidade de reinício pela tecla **R**.

Para além do exigido, foram acrescentadas funcionalidades que enriquecem a aplicação: um sistema de menus com máquina de estados e garagem para personalização das viaturas, um motor de áudio baseado na *Web Audio API* com *crossfade* entre temas, e um painel de controlo *lil-gui* que expõe em tempo real os parâmetros do sistema. A organização modular do código, com responsabilidades bem delimitadas, foi decisiva para permitir o desenvolvimento em paralelo e simplificar a deteção de problemas.

Entre as principais dificuldades destaca-se o equilíbrio entre densidade visual e desempenho, mitigado com *InstancedMesh* para elementos repetidos e com a calibração dos *shadow maps* por bioma, e a construção das viaturas exclusivamente com *BoxGeometry* e *CylinderGeometry*, que exigiu soluções como o guarda-lamas facetado da mota e os anéis concêntricos do *hover-skate*, reforçando o domínio das transformações geométricas e da composição hierárquica.

Como evolução futura, identificam-se três vetores principais: a introdução de adversários controlados por inteligência artificial e eventual extensão a multijogador *online*; a expansão do conteúdo com novas arenas e viaturas, sendo que *glider.js* e *speeder.js* já constituem ponto de partida; e a introdução de mecânicas adicionais como poderes recolhíveis ou modos de jogo alternativos. Em síntese, o **NEON DRIVE** constituiu um exercício completo de aplicação dos conceitos da Computação Gráfica, articulando *pipeline* gráfico, modelação por primitivas, transformações geométricas, iluminação, mapeamento de texturas e interação em tempo real.